

Rapport TP1 — Mon premier service web SOAP en Java

Projet : Service Web SOAP en Java

Nom du projet : serviceSoapExemple1

Technologies utilisées : Java 11, Maven, JAX-WS, IntelliJ IDEA, SoapUI

Réalisé par : RAKOTOARISON

Formation : Hasina

Année : 2024-2025

Introduction — SOAP, le facteur des applications

Dans ce projet, j'ai créé un petit service web SOAP en Java.

L'idée principale est simple : permettre à deux applications de communiquer entre elles, même si elles ne sont pas écrites dans le même langage.

On peut imaginer SOAP comme un **facteur numérique** :

- le client prépare une demande dans une enveloppe XML ;
- le serveur reçoit cette enveloppe ;
- il lit la demande ;
- il exécute l'opération demandée ;
- puis il renvoie une réponse, elle aussi sous forme XML.

SOAP signifie **Simple Object Access Protocol**.

C'est un protocole basé sur XML, souvent utilisé avec HTTP. Dans notre cas, il sert à appeler des méthodes Java à distance comme si elles étaient disponibles depuis une autre application.

1. Objectif du projet

Le but de ce TP est de comprendre comment créer, publier et tester un service web SOAP.

Le service réalisé permet notamment de :

- convertir une valeur numérique ;
- additionner deux nombres ;

- exposer ces opérations sous forme de service web ;
- générer automatiquement un document WSDL ;
- tester le service avec SoapUI.

Le projet n'est donc pas seulement un programme Java classique.
Il devient un petit serveur capable de recevoir des requêtes SOAP.

2. Organisation du projet

Le projet a été créé avec Maven afin de mieux gérer les dépendances nécessaires au fonctionnement de SOAP avec Java 11.

L'arborescence principale est la suivante :

```
serviceSoapExemple1/
├── src/
│   ├── main/
│   │   └── java/
│   │       ├── org/
│   │       │   └── example/
│   │       │       ├── Application.java
│   │       │       ├── Main.java
│   │       │       └── MonserviceWeb.java
│   └── test/
├── pom.xml
├── .gitignore
└── target/
```

Le dossier le plus important est :

`src/main/java/org/example/`

C'est ici que se trouvent les classes Java du projet.

3. Maven : la boîte à outils du projet

Avec Java 11, certaines bibliothèques liées aux services SOAP ne sont plus intégrées directement dans le JDK.

J'ai donc utilisé Maven pour ajouter les dépendances nécessaires.

Le fichier `pom.xml` joue le rôle de **liste de courses technique** : il indique à Maven quelles bibliothèques télécharger pour que le projet fonctionne.

Exemple de dépendances utilisées :

```
<dependencies>
  <dependency>
```

```

    <groupId>javax.jws</groupId>
    <artifactId>javax.jws-api</artifactId>
    <version>1.1</version>
</dependency>

<dependency>
    <groupId>javax.xml.ws</groupId>
    <artifactId>jaxws-api</artifactId>
    <version>2.3.1</version>
</dependency>

<dependency>
    <groupId>com.sun.xml.ws</groupId>
    <artifactId>jaxws-rt</artifactId>
    <version>2.3.7</version>
</dependency>
</dependencies>

```

Grâce à ces dépendances, les annotations comme `@WebService`, `@WebMethod` et `@WebParam` sont reconnues par le projet.

Capture à insérer ici : fichier `pom.xml` dans IntelliJ.

4. La classe `MonServiceWeb` : le cœur du service

La classe `MonServiceWeb.java` contient les opérations proposées par le service SOAP.

```

package org.example;

import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.jws.WebService;

@WebService(targetNamespace = "http://www.polytech.fr")
public class MonServiceWeb {

    @WebMethod(operationName = "convertir")
    public double conversion(double mt) {
        return mt * 0.9;
    }

    public double somme(@WebParam(name = "parametre_1") double a,
                       @WebParam(name = "parametre_2") double b) {
        return a + b;
    }
}

```

Cette classe est annotée avec :

@WebService

Cette annotation permet d'indiquer que la classe devient un service web SOAP.

La méthode conversion permet de transformer une valeur en la multipliant par 0.9.

La méthode somme permet d'additionner deux nombres.

Les paramètres sont nommés avec @WebParam, ce qui permet d'avoir des noms plus clairs dans les messages SOAP.

Capture à insérer ici : classe MonserviceWeb.java.

5. La classe Application : le bouton ON du service

La classe Application.java sert à publier le service localement.

```
package org.example;

import javax.xml.ws.Endpoint;

public class Application {

    public static void main(String[] args) {

        System.out.println("Début de déploiement de mon service");

        String url = "http://localhost:8888/";
        Endpoint.publish(url, new MonserviceWeb());

        System.out.println("Le service web est déployé");
        System.out.println("WSDL : " + url + "?wsdl");
    }
}
```

La ligne la plus importante est :

```
Endpoint.publish(url, new MonserviceWeb());
```

Elle permet de lancer le service web sur l'adresse :

http://localhost:8888/

Une fois le programme lancé, le service reste actif tant que l'application Java continue de tourner.

Capture à insérer ici : classe Application.java.

6. Le WSDL : la carte d'identité du service

Après le lancement du programme, le WSDL est accessible dans le navigateur avec l'adresse :

`http://localhost:8888/?wsdl`

Le WSDL est un document XML qui décrit le service.

Il indique notamment :

- le nom du service ;
- les opérations disponibles ;
- les paramètres attendus ;
- le type des réponses ;
- l'adresse du service.

On peut comparer le WSDL à une **notice d'utilisation automatique**.

Un client SOAP peut le lire pour savoir comment communiquer avec le service.

Capture à insérer ici : page WSDL affichée dans le navigateur.

7. Test avec SoapUI : discussion avec le service

Pour vérifier que le service fonctionne, j'ai utilisé SoapUI.

Les étapes de test sont les suivantes :

1. Lancer la classe `Application.java` dans IntelliJ.
2. Ouvrir SoapUI.
3. Créer un nouveau projet SOAP.
4. Coller l'adresse du WSDL :

`http://localhost:8888/?wsdl`

5. Laisser SoapUI générer les requêtes.
6. Modifier les valeurs dans la requête.
7. Envoyer la requête.
8. Observer la réponse du service.

Exemple pour l'opération `somme` :

- valeur 1 : 5
- valeur 2 : 3
- réponse attendue : 8

Exemple pour l'opération `conversion` :

- valeur envoyée : 100
- réponse attendue : 90

SoapUI permet donc de vérifier que le client et le serveur arrivent bien à communiquer grâce au protocole SOAP.

Capture à insérer ici : requête et réponse dans SoapUI.

8. Difficultés rencontrées

Pendant le projet, j'ai rencontré plusieurs problèmes.

Le premier problème concernait Java.

Le projet utilisait Java 11, mais certaines bibliothèques SOAP ne sont plus disponibles directement dans cette version. Il a donc fallu ajouter les dépendances dans Maven.

Le deuxième problème concernait le fichier `pom.xml`.

Au début, les dépendances étaient placées au mauvais endroit. Elles doivent obligatoirement être à l'intérieur de la balise `<project>`.

Le troisième problème concernait l'import de `Endpoint`.

La classe `Application.java` avait besoin de :

```
import javax.xml.ws.Endpoint;
```

Une fois ces erreurs corrigées, le service a pu être lancé correctement.

9. Conclusion

Ce projet m'a permis de comprendre comment un service web SOAP fonctionne en Java.

J'ai appris à :

- créer un projet Maven ;
- ajouter des dépendances SOAP ;
- utiliser les annotations JAX-WS ;
- publier un service localement ;
- consulter un WSDL ;
- tester les opérations avec SoapUI ;
- préparer le projet pour GitHub.

Même si SOAP peut sembler ancien par rapport aux API REST, il reste intéressant car il montre clairement comment les applications peuvent échanger des messages structurés en XML.

Ce projet m'a aussi permis de mieux comprendre le rôle de Maven et l'importance des dépendances dans un projet Java.

Glossaire

SOAP

Protocole qui permet à deux applications de discuter avec des messages XML.

XML

Format de texte structuré avec des balises. Il ressemble un peu au HTML, mais il sert surtout à organiser des données.

WSDL

Document qui décrit le service SOAP. C'est comme le menu d'un restaurant : il indique ce que le service propose.

JAX-WS

Bibliothèque Java qui permet de créer des services web SOAP avec des annotations.

Maven

Outil qui gère les dépendances et la construction du projet Java.

Endpoint

Point d'entrée du service. C'est l'adresse où le client peut contacter le service.

SoapUI

Logiciel qui permet de tester des services SOAP sans écrire un client complet.